
Arquiteturas paralelas

Sumário

1.1	Introdução	20
1.2	Classificação de Flynn	22
1.2.1	Arquiteturas SISD	23
1.2.2	Arquiteturas SIMD	23
1.2.3	Arquiteturas MIMD	27
1.3	Aceleradores	31
1.3.1	Conceitos gerais	31
1.3.2	GPUs	34
1.4	Exercícios	42

Antes de abordar os conceitos de programação paralela, é necessário compreender os fundamentos e a organização das arquiteturas paralelas, incluindo a dos aceleradores, responsáveis por executar os modelos de programação apresentados ao longo deste livro.

1.1 INTRODUÇÃO

No estudo do paralelismo, é necessário levar em conta os possíveis níveis de execução de uma aplicação:

- **nível de instrução (granulosidade fina):** encontrado em processadores com *pipeline*, superescalares ou VLIW (*Very Long Instruction Word*). Esse tipo de paralelismo, transparente ao usuário, é normalmente explorado pelo compilador ou diretamente pelo *hardware*, com a execução simultânea de várias instruções nas diversas unidades funcionais de um processador.

- **nível de *thread* (granulosidade média):** encontrado em variadas arquiteturas de processador, destacando-se as *multithreading*, de *multithreading* simultâneo (SMT – *Simultaneous Multithreading processor*), *multicore* e aceleradores. Neste caso, são necessárias várias *threads* executando em paralelo, necessitando da intervenção do programador ou do compilador para o aproveitamento desse tipo de paralelismo.

- **nível de processo (granulosidade grossa):** encontrado em multiprocessadores (memória compartilhada) e multicomputadores (memória distribuída), que normalmente se utilizam de processos como unidade de computacional, e requerem a também a intervenção explícita do programador e o suporte de bibliotecas e/ou ambientes de execução paralelos.

As arquiteturas paralelas são aquelas capazes de explorar o paralelismo existente nas aplicações nesses diversos níveis e podem ser organizadas de

diversas maneiras. Para um melhor entendimento dos paradigmas de programação ilustrados no livro, são apresentadas algumas formas de organização dessas arquiteturas paralelas.

As arquiteturas que exploram o paralelismo no nível de instrução não irão receber nossa atenção, pois isso é feito automaticamente pelo *hardware* ou compilador, sem precisar da intervenção do programador.

Entre as arquiteturas que exploram o paralelismo no nível de *threads* ou processos, três tipos serão estudados com maior detalhe: multiprocessadores, multicomputadores e aceleradores.

Os **multiprocessadores**, conhecidos também como *multicore* ou *chip multiprocessing* (CMP), são caracterizados pela existência de diversos processadores (núcleos) em um mesmo encapsulamento, compartilhando algum nível de memória cache e a memória principal. Eventualmente, este tipo de arquitetura pode ser expandido para vários *chips*, cada um com vários núcleos, compartilhando uma mesma memória global.

Notem que, em quaisquer dos casos, a comunicação entre as *threads* é feita através de variáveis na memória compartilhada (ou na memória cache no caso dos *multicore*), necessitando da intervenção explícita do programador para a coordenação do paralelismo. O **OpenMP** é o *framework* básico para a exploração deste tipo de paralelismo.

Os **multicomputadores**, também conhecidos como *clusters*, são caracterizados pela presença de diversos nós computacionais, onde cada nó é composto por um ou mais processadores, cada um possuindo sua própria memória local. Diferentemente dos multiprocessadores, onde há uma memória compartilhada, nos multicomputadores não existe uma memória global acessível a todos os nós diretamente.

Cada nó mantém sua independência e a comunicação entre os nós ocorre através de um mecanismo de troca de mensagens. Essa troca de mensagens pode ser realizada utilizando redes de alta velocidade que conectam os nós, como InfiniBand, Ethernet ou outras tecnologias específicas. Esse modelo

requer que o programador implemente explicitamente a comunicação e a coordenação entre os nós com uso de bibliotecas de comunicação.

Esse modelo torna a comunicação mais complexa, pois exige a sincronização das mensagens e a garantia de consistência dos dados entre os processos. Além disso, o desempenho de um multicomputador depende da latência e largura de banda da rede de comunicação utilizada. O **MPI** é amplamente utilizado para explorar o paralelismo em multicomputadores, fornecendo abstrações eficientes para troca de mensagens e sincronização de tarefas.

As arquiteturas com **aceleradores** são um tipo particular de exploração de paralelismo no nível de *thread*, onde trechos computacionalmente intensivos do programa, chamados de *kernels* são enviados para execução nos aceleradores, cuja memória é separada daquela do hospedeiro. Isso requer que tanto o código como os dados sejam transferidos da memória do hospedeiro para a memória do acelerador através de barramentos dedicados de alta velocidade. Os tipos de aceleradores mais utilizados são os *manycores*; as FPGAs; e as GPUs (*Graphics Processing Unit*), cujo paralelismo pode ser adequadamente explorado com uso de bibliotecas de programação como **OpenMP offloading** e o **OpenACC**.

1.2 CLASSIFICAÇÃO DE FLYNN

Antes de detalhar essas arquiteturas, apresentamos a classificação de Flynn (FLYNN, 1966), que distingue os diferentes fluxos de um programa em execução:

- **Single Instruction Stream (SI)**
- **Multiple Instruction Streams (MI)**
- **Single Data Stream (SD)**
- **Multiple Data Streams (MD)**

A associação desses diversos tipos de fluxos dá origem às seguintes categorias de organização dos computadores:

- **SISD** – Single Instruction Stream Single Data Stream – (p.ex. processadores convencionais, *pipelined* ou superescalares)
- **SIMD** – Single Instruction Stream Multiple Data Streams – (p.ex. processadores vetoriais, unidades funcionais vetoriais, arquiteturas SIMD)
- **MIMD** – Multiple Instruction Streams Multiple Data Streams – (p.ex. multiprocessadores, multicomputadores)

Alguns detalhes desses tipos de arquitetura são apresentados seguir.

1.2.1 Arquiteturas SISD

São os processadores convencionais, onde apenas um processo ou *thread* é executado por vez, podendo fazer uso de técnicas de exploração de paralelismo temporal (*pipeline*) ou espacial (superescalares) em *hardware*.

Nesse caso, o paralelismo, se explorado, é feito de forma transparente ao usuário, no nível de instrução, com um suporte mínimo do compilador, por exemplo, no escalonamento mais adequado das instruções em linguagem de máquina para a execução em várias unidades funcionais.

Como já colocado, este tipo de arquitetura não irá receber nossa atenção, pois a exploração de paralelismo é feita automaticamente pelo *hardware* ou compilador, sem necessidade de que o programador faça modificações no código do programa.

1.2.2 Arquiteturas SIMD

As arquiteturas SIMD consistem basicamente de uma única instrução operando sobre diversos dados diferentes, como ocorre, por exemplo, nos processadores vetoriais, cujos exemplos mais famosos são os supercomputadores da linha Cray ([Wikipedia, 2020a](#)).

Este tipo de arquitetura se subdivide em diversos outros tipos, a destacar:

- **Arquiteturas SIMD convencionais**
- **Processadores vetoriais**
- **Arquiteturas sistólicas**

Arquiteturas SIMD convencionais

É uma classe importante de processadores, devido a fatores como:

- Simplicidade de conceitos e programação;
- Regularidade da estrutura;
- Facilidade de escalabilidade em tamanho e desempenho; e
- Aplicação direta em uma série de aplicações paralelas para obter o desempenho necessário.

Os processadores executam sincronizadamente a mesma instrução sobre dados diferentes. Utiliza vários processadores especiais muito mais simples, organizados em geral de forma matricial. É muito eficiente em aplicações onde cada processador pode ser associado a uma sub-matriz independente de dados (processamento de imagens, alguns algoritmos matemáticos específicos, etc.).

Um exemplo clássico é o processador ILLIAC IV, que funcionou entre 1967 e 1972 (BARNES et al., 1968). Modernamente, é raro encontrar arquiteturas desse tipo, que são restritas a alguns tipos de *hardware* especializado para aplicações como processamento digital de sinais (DSP), processamento de imagem e em alguns aceleradores de *hardware*, onde a natureza das tarefas se beneficia diretamente da execução da mesma operação em múltiplos pontos de dados.

Processadores vetoriais

Os processadores vetoriais são arquiteturas *pipelined* do tipo SIMD, ou seja, uma única instrução opera sobre vários dados. Um processador vetorial possui instruções para operar em vetores: um conjunto linear de valores. Uma operação vetorial típica pode, por exemplo, adicionar dois vetores com N elementos em notação ponto flutuante para obter um único vetor de resultado com N elementos.

A instrução vetorial é equivalente a um laço inteiro, onde cada iteração calcula vários elementos do resultado, atualiza os índices e retorna para o início. As instruções vetoriais possuem as seguintes características:

- Cada instrução equivale a um laço completo;
- O cálculo de cada resultado não depende de resultados anteriores, assim é possível o uso de *pipelines* profundos sem a ocorrência de dependências de dados; e
- O padrão de acesso à memória para a busca dos operandos é conhecido e regular, se beneficiando da utilização de memória com entrelaçamento (*interleaving*) ou mesmo memórias caches.

A maioria dos processadores modernos possui instruções vetoriais em seu conjunto de instruções. Um exemplo típico são as instruções AVX-512 da Intel ou AVX2 da AMD. Na maior parte dos casos, a exploração deste tipo de paralelismo, em diversas linguagens de programação, é feita automaticamente pelo compilador. Mas, em alguns casos, pode ser necessária a intervenção do programador para otimizar a geração de código, como, por exemplo, a indicação do comprimento ideal de vetorização ou dos níveis de laços que podem ser colapsados. Veja maiores detalhes na Seção 3.8.

Arquiteturas sistólicas

Arquiteturas sistólicas foi um termo utilizado por H.T. Kung, em 1978, para definir um tipo de arquitetura na qual os dados são “bombeados” através de uma estrutura em *hardware*, em analogia com o funcionamento do coração (KUNG, 1982).

Uma arquitetura sistólica consiste em um conjunto de células interconectadas, cada uma delas capaz de realizar uma operação simples. As células se comunicam diretamente com seus vizinhos, formando uma estrutura de processamento em fluxo, onde os dados “pulsam” (daí o termo *sistólico*, em analogia ao sistema circulatório). O uso de estruturas de comunicação e controle simples e regulares oferece uma grande vantagem em relação às arquiteturas convencionais, pois permite a execução de operações em paralelo de maneira altamente eficiente, reduzindo a complexidade do *hardware* e maximizando seu uso.

Tensor Processing Unit (TPU) do Google

As TPUs do Google, são usadas para acelerar tarefas de aprendizado profundo e incluem *arrays sistólicos* como parte de sua arquitetura, para realizar operações de multiplicação e soma de matrizes (GEMM – *General Matrix Multiply*), uma tarefa comum em redes neurais.

Os dados (pesos e entradas) fluem pelos *arrays sistólicos* em uma organização de células interconectadas, permitindo o cálculo eficiente de operações matriciais de grande escala, pois o fluxo de dados contínuo reduz o tempo de espera entre operações. Minimizam ainda a movimentação de dados entre memória e processador, reduzindo o consumo de energia e aumentando a velocidade do processamento. As arquiteturas sistólicas ainda encontram aplicações em sistemas especializados, particularmente em áreas onde o processamento massivo e paralelo é fundamental, como inteligência artificial (IA) e aprendizado de máquina (ML – *Machine Learning*).

1.2.3 Arquiteturas MIMD

As arquiteturas MIMD se subdividem em dois grandes grupos:

- **Arquiteturas de Memória Distribuída** – cada processador é capaz de endereçar apenas a sua própria memória local.
- **Arquiteturas de Memória Compartilhada** – cada processador consegue ter acesso a um espaço de endereçamento global do sistema.

MIMD com memória distribuída

Neste tipo de arquitetura, cada processador enxerga apenas o seu espaço de memória. Um exemplo dessa arquitetura pode ser visto na Figura 1.1. Apresenta como vantagens serem altamente escaláveis e permitirem a construção de processadores maciçamente paralelos. As suas grandes desvantagens são a necessidade de se fazer uma boa distribuição de carga entre os processadores, quer seja automaticamente, quer seja manualmente.

Nesse tipo de arquitetura, como os *clusters* de multicomputadores, onde cada nó possui sua memória local, a troca de mensagens resolve tanto o problema da comunicação dos processos como da sua sincronização. É precisamente essa característica que estabelece o MPI (*Message Passing Interface*) como o padrão de fato para a programação paralela nesse tipo de sistema, focando na comunicação entre processos.

Entre outras desvantagens deste tipo de arquitetura, estão a dificuldade de sobrepor o processamento da computação com a comunicação entre processos e a necessidade de se evitar as situações de *deadlock*, tanto no nível de aplicação como no nível de sistema operacional, possíveis de ocorrer quando há espera circular pelo envio e recebimento de mensagens. Além disso, é um modelo de programação menos natural, demandando diversas modificações em relação ao código convencional sequencial.

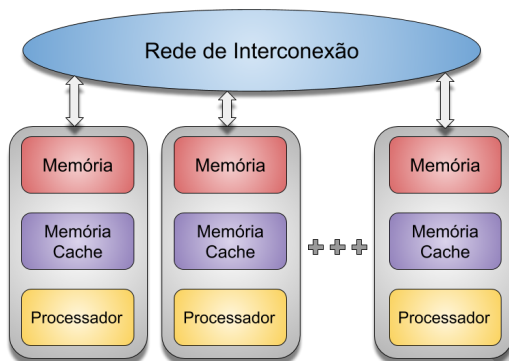


Figura 1.1 – Arquiteturas MIMD com Memória Distribuída

MIMD com memória compartilhada

Nesse tipo de arquitetura, cada processador tem acesso a um espaço de endereçamento global comum a todos. Um exemplo dessa arquitetura é apresentada na Figura 1.2. Apresenta como vantagem não necessitar do particionamento de código ou dados, logo técnicas de programação sequenciais podem ser facilmente adaptadas. Não há também necessidade da movimentação física dos dados, quando dois ou mais processadores se comunicam, resultando em uma comunicação entre processos ou *threads* bastante eficiente.

As suas desvantagens são a necessidade do uso de primitivas especiais de sincronização quando do acesso a regiões compartilhadas de memória, para assegurar um resultado correto para a computação, além da falta de escalabilidade devido ao problema de contenção de memória. Depois de um determinado número de processadores, a adição de mais processadores não aumenta o desempenho.

Para gerenciar o paralelismo nesse ambiente e coordenar o acesso à

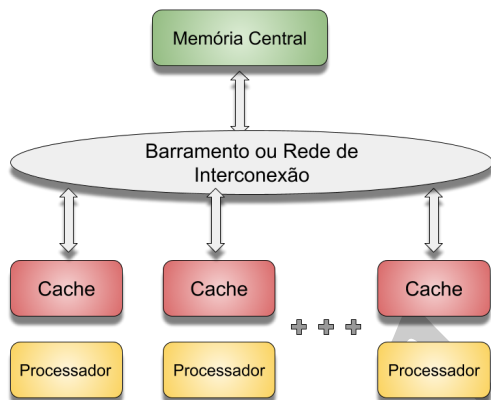


Figura 1.2 – Arquiteturas MIMD com Memória Compartilhada

memória compartilhada, o OpenMP emerge como o *framework* básico, extraíndo paralelismo natural dos laços e permitindo a sincronização adequada no acesso às variáveis compartilhadas com o uso, por exemplo, de operações atômicas e de redução.

As arquiteturas em questão apresentam diferentes subclassificações e variações em sua organização, das quais algumas são discutidas a seguir.

Arquiteturas UMA

As arquiteturas UMA (*Uniform Memory Access*) caracterizam-se por todos os processadores possuírem o mesmo tempo de acesso a uma memória global única, geralmente interconectados por um barramento compartilhado. Para mitigar a contenção nesse barramento, processadores comerciais modernos empregam um ou dois níveis de cache exclusivos para cada núcleo, incluindo uma cache compartilhada de nível 3 para comunicação inter-núcleos, evitando acessos frequentes ao barramento e à memória principal. A utilização de caches, por sua vez, requer algoritmos de coerência de cache implementa-

dos em *hardware*, como a técnica de *snooping* para manter a consistência dos dados nas caches locais dos processadores.

Um dos grandes problemas dos sistemas de memória compartilhada com uso de barramento único é a escalabilidade. Ou seja, para um grande número de processadores a contenção no uso do barramento aumenta até o ponto no qual, mesmo com o uso da cache, a adição de novos processadores não é mais efetiva. Um barramento único tem um tamanho (comprimento físico) limitado, o que também diminui o número máximo de processadores e módulos de memória que podem ser acrescentados ao sistema.

Ao invés do barramento, pode-se utilizar uma rede de interconexão com alto *throughput* e baixa latência. Essa rede tem características especiais que permitem o acesso simultâneo de vários processadores a módulos de memória distintos. Uma solução utilizada para permitir que a quantidade de memória do sistema cresça à medida que cresce o número de processadores é agregar um módulo de memória local para cada processador do sistema. Esses sistemas são chamados de MIMD com memória compartilhada distribuída, ou ainda, arquiteturas com tempo de acesso não uniforme à memória (*Non-uniform Memory Access* – NUMA), conforme apresentado a seguir.

Arquiteturas NUMA

Arquiteturas NUMA (*Non-Uniform Memory Access*) organizam a memória em blocos, cada qual conectado localmente a um processador, formando um nó. Esses nós interconectam-se por redes com diversas topologias e protocolos, visando acesso eficiente aos dados remotos. O acesso à memória local é significativamente mais rápido que o acesso remoto.

Essa disparidade de tempo de acesso exige uma programação cuidadosa, de modo a tolerar e ocultar a latência no acesso a memórias distantes. Uma rede de interconexão de alto desempenho é utilizada para minimizar o tempo de acesso remoto e melhorar a escalabilidade. Técnicas como a busca antecipada de dados/páginas também contribuem para reduzir a latência.

O emprego de caches locais, que armazenam cópias de dados remotos, mitiga a contenção e a latência, evitando a travessia da rede de interconexão. No entanto, isso impõe a necessidade de algoritmos complexos para manter a consistência dos dados entre as múltiplas caches locais.

Modernamente, como exemplos de sistemas que utilizam a arquitetura NUMA, estão os servidores com placas mães com vários processadores Intel Xeon ou AMD EPYC, onde cada soquete de processador possui múltiplos *links* de alta velocidade para conectá-lo diretamente a outros soquetes na placa-mãe com uso de diversas topologias.

Apesar da natureza de memória compartilhada, arquiteturas NUMA modernas frequentemente oferecem bibliotecas de troca de mensagens como o MPI para comunicação entre núcleos. Nesses casos, em vez de enviar mensagens pela rede, apenas ponteiros são copiados entre processos, permitindo acesso direto aos módulos de memória onde a informação reside.

A programação em arquiteturas NUMA frequentemente se beneficia de modelos híbridos, combinando de modo eficaz o uso de MPI para comunicação entre nós com modelos de programação paralela de memória compartilhada dentro de cada nó (como OpenMP). Isso permite explorar a localidade da memória dentro do nó e usar a troca de mensagens para o compartilhamento de dados entre os nós.

1.3 ACELERADORES

1.3.1 Conceitos gerais

Uma opção importante para obter um alto grau de paralelismo com baixo consumo de energia são os aceleradores. Aceleradores são dispositivos especiais de *hardware* que trabalham em conjunto com processadores convencionais, executando trechos intensivos de código, com alto potencial de paralelismo, chamados de *kernels*.

Uma característica particular desses sistemas é que os aceleradores geral-

mente possuem uma memória própria, distinta da memória do hospedeiro. Isso significa que o espaço de endereçamento da memória do hospedeiro e do acelerador são separados. Para que o acelerador execute esses *kernels*, é necessário transferir tanto o código quanto os dados da memória do hospedeiro para a memória do acelerador. Essa movimentação de dados ocorre através de barramentos dedicados de alta velocidade, mas, em geral, o barramento é mais lento comparado à largura de banda das memórias dos aceleradores.

Então, a movimentação de dados é um fator de grande impacto no desempenho e pode ser um desafio gerenciar corretamente. Estratégias como minimizar a transferência de dados desnecessários, usar alocação direta no acelerador, quando possível, ou mapear apenas as regiões de dados realmente usadas, são importantes para otimizar o desempenho.

Técnicas como sobrepor a cópia de dados com outras operações também podem reduzir a penalidade de desempenho associada às transferências. Embora exista o conceito de Memória Unificada (*Unified Memory*) que oferece um espaço de endereçamento único acessível tanto pela CPU quanto pela GPU, simplificando a programação, o gerenciamento subjacente ainda pode ter implicações de desempenho.

Além disso, os aceleradores possuem sua própria hierarquia de memória interna. Embora a estrutura exata varie entre arquiteturas, é comum que existam diferentes níveis (como registradores, caches e memória compartilhada) dentro do acelerador para proporcionar acesso rápido aos dados.

Entre esses, destacam-se os seguintes tipos de dispositivo: processadores *manycores*; *Field Programmable Gate Arrays* (FPGAs); e *Graphics Processing Units* (GPUs) – observe a Figura 1.3.

Os **processadores *manycore*** são projetados para um alto grau de processamento paralelo, contendo vários núcleos de processadores simples e independentes (variando de algumas dezenas de núcleos a milhares). Os processadores *manycore* são utilizados tanto em sistemas embarcados como em computação de alto desempenho ([Wikipedia, 2020b](#)).

Para as cargas de trabalho orientadas para a vazão, pode-se conseguir mais desempenho/potência e desempenho/área do *chip*, levando-se ao extremo o conceito de que a latência não é importante, e construindo-se um processador com muitos núcleos pequenos ao invés de poucos núcleos grandes. Existem diversos exemplos desse tipo de arquitetura, entre os quais destacam-se o processador Xeon Phi da Intel e o processador SW26010, do supercomputador SunWay TaihuLight (RAHMAN, 2013).

As **Field Programmable Gate Arrays** (FPGAs) são dispositivos de *hardware* programáveis por *software*. Ou seja, podem ser personalizados para executar com eficiência determinados trechos de código com uso intensivo de dados e/ou alta demanda computacional. Assim, esses dispositivos podem ser programados e configurados de acordo com o tipo de aplicação executada, se adaptando assim à solução de diversos tipos de problemas.

Por exemplo, as FPGAs podem ser programadas para implementar arquiteturas sistólicas específicas em diversas aplicações, incluindo processamento de sinais digitais (DSP), aceleração de algoritmos de criptografia e simulações científicas que envolvem álgebra linear. Os dispositivos FPGAs da Xilinx e Intel são frequentemente utilizados para criar essas arquiteturas sistólicas otimizadas para tarefas específicas.



Figura 1.3 – Tipos de arquiteturas: multicore e acelerador

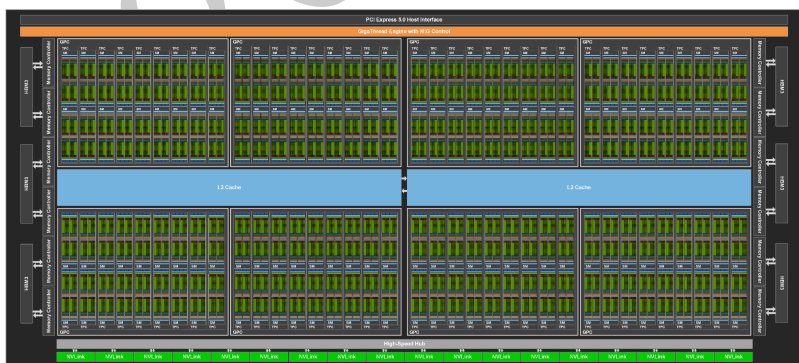
A **GPU** (*Graphical Processing Unit*), desenvolvida pela NVIDIA nos anos 90, se baseia em um grande número de núcleos para processamento paralelo maciço com foco na eficiência energética e para aplicações com demandas que melhorem o *throughput*. Inicialmente projetados com o objetivo de atender à área de jogos, rapidamente mostrou-se muito eficiente na execução de vários tipos de aplicações científicas (ZANOTTO et al., 2012).

1.3.2 GPUs

As arquiteturas dos aceleradores gráficos (GPUs) são bem diferenciadas das arquiteturas dos processadores convencionais. O paralelismo nos aceleradores gráficos é explorado através de um conjunto maciço de multiprocessadores de fluxo (*streaming multiprocessors* – SM), executando em paralelo e de forma sincronizada trechos computacionalmente intensivos, chamados de *kernels*, das diversas aplicações.

Um melhor entendimento do funcionamento dos aceleradores gráficos (GPUs) pode ser alcançado, sem perda de generalidade, pela análise da arquitetura Hopper (NVIDIA, 2022), desenvolvida pela NVIDIA.

Figura 1.4 – Arquitetura NVIDIA Hopper



A Figura 1.4 apresenta diversos níveis de hierarquia de memória, alguns compartilhados e outros exclusivos de cada multiprocessador de fluxo (SM),

que caracterizam a arquitetura diferenciada do acelerador gráfico. Esses e outros aspectos são analisados a seguir.

Principais características da arquitetura Hopper

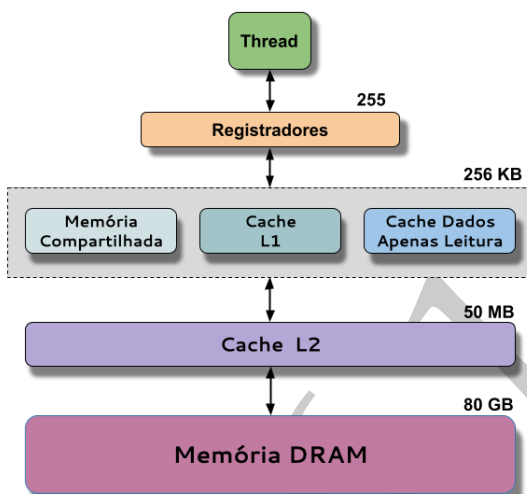
Cada unidade de multiprocessamento (SM) possui 128 núcleos CUDA de precisão simples e 64 de precisão dupla, com cada núcleo dedicado à execução de operações matemáticas envolvendo números inteiros e cálculos de ponto flutuante. Esses núcleos operam em um *pipeline* paralelo, permitindo a execução simultânea de múltiplas operações, o que aumenta consideravelmente a eficiência e a velocidade de processamento. A arquitetura foi projetada com foco na relação desempenho/consumo de energia, um aspecto essencial para a computação de alto desempenho moderna, onde o uso eficiente de energia é fundamental para maximizar a capacidade de processamento dentro de limites ambientalmente sustentáveis. Uma unidade SM da arquitetura Hopper denominada H100 pode ser vista na Figura 1.5.

O escalonador do multiprocessador de fluxo (SM) dispara as *threads* em grupos de 32 *threads* chamadas de *warps*. Cada SM possui quatro escalonadores de *warp*, permitindo um máximo de quatro *warps* disparadas e executadas concorrentemente. O número de registradores pode chegar até 255 registradores utilizados simultaneamente por cada *thread*.

Para melhorar ainda mais o desempenho, a arquitetura Hopper possui 32 instruções *shuffle*. Essas instruções são usadas para otimizar operações de troca de dados entre *threads* dentro de um *warp*, melhorando a eficiência e o desempenho das operações paralelas, impactando em muito o desempenho de aplicações como, por exemplo, a transformada de Fourier (FFT).

Outro tipo de instrução disponível são as operações atômicas em memória, permitindo que as *threads* realizem adequadamente operações de *read-modify-write*, como soma, máximo, mínimo e compare-e-troque em estruturas de dados compartilhadas. As operações atômicas permitem a ordenação e o acesso em paralelo a estruturas de dados compartilhadas sem

Figura 1.6 – Hierarquia de Memória



entre cache L1 e memória compartilhada, com esta última suportando até 228 KB. Esse espaço de armazenamento flexível permite aos desenvolvedores personalizá-lo conforme as necessidades específicas de cada aplicação. A memória compartilhada pode ser dividida em até 8 bancos de memória, permitindo acessos paralelos por múltiplas *threads*. Essa divisão é projetada para melhorar a eficiência das operações, reduzindo a latência e aumentando a largura de banda, o que contribui para um desempenho geral mais rápido e eficiente.

Além da cache L1, a arquitetura Hopper possui uma cache apenas de leitura de até 64 MB. O gerenciamento dessa cache pode ser feito automaticamente pelo compilador ou explicitamente pelo programador. O acesso a uma variável ou estrutura de dados que o programador identifica como apenas de leitura, pode ser declarada com a palavra chave `const __restrict`, permitindo ao compilador carregá-la na cache apenas de leitura¹.

¹ <https://chipsandcheese.com/p/nvidias-h100-funny-l2-and-tons-of-bandwidth>

A arquitetura Hopper possui também uma cache L2 de 50 MB que permite um melhor fluxo de dados para as cargas de trabalho de processamento intensivo característicos de aplicações de IA e HPC. A cache L2 é o ponto primário de unificação de dados entre os diversos SMs, servindo operações de *load*, *store* e de textura, provendo um compartilhamento de dados eficiente e de alta velocidade.

Algoritmos onde o endereço dos dados é conhecido previamente, tais como solucionadores de física, *ray tracing* e multiplicação esparsa de matrizes, se beneficiam especialmente das hierarquia de cache. Os *kernels* de filtro e convolução onde é necessário que diversos SMs leiam os mesmos dados, também se beneficiam dessa hierarquia.

Finalmente, no nível mais alto da hierarquia encontra-se a memória global compartilhada, com até 80 GB e oferece largura de banda entre 2.000 e 3.000 GB/s, conforme o tipo de memória empregado (HBM2e ou HBM3).

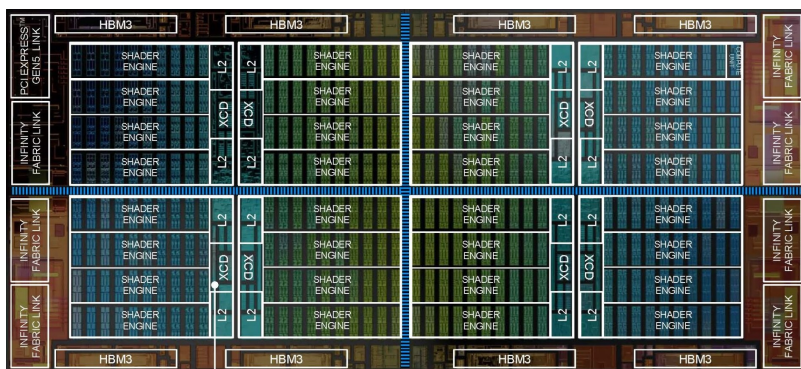
A arquitetura possui uma série de outras facilidades como código de correção de erro, paralelismo dinâmico, gerenciamento de filas de trabalho e unidade de gerenciamento de *grids*, que melhoram o desempenho e a confiabilidade do acelerador. Maiores detalhes podem ser vistos na referência (NVIDIA, 2022).

Principais características da arquitetura AMD Instinct

A AMD Instinct MI300X é uma unidade de processamento gráfico (GPU) projetada para computação de alto desempenho, especialmente em ambientes de *data center* e inteligência artificial. A MI300X utiliza a arquitetura CDNA 3, otimizada para cargas de trabalho de computação e aprendizado de máquina, e é equipada com um número elevado de unidades de computação, permitindo executar múltiplas operações em paralelo, o que é importante para tarefas intensivas em dados (AMD, 2024).

A Figura 1.7 apresenta os diversos níveis de hierarquia de memória do acelerador AMD Instinct.

Figura 1.7 – Arquitetura AMD Instinct



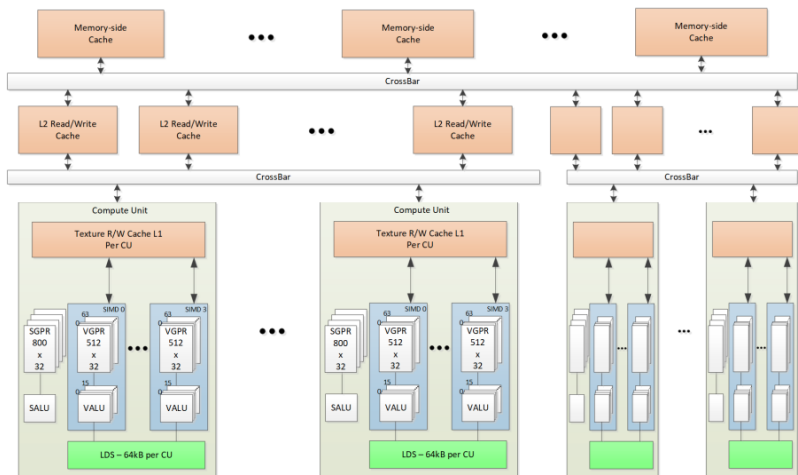
A composição da memória da GPU AMD Instinct MI300X é um dos seus pontos mais fortes e é feita de forma bastante avançada para atender às demandas de cargas de trabalho de IA e HPC.

A MI300X utiliza 192 GB de memória HBM3 (*High Bandwidth Memory 3*). A HBM é um tipo de memória RAM de alto desempenho que é empilhada verticalmente (em 3D) e conectada diretamente ao processador gráfico. Isso permite uma largura de banda de memória muito maior em comparação com as memórias GDDR tradicionais.

A arquitetura CDNA 3 da MI300X adota um projeto baseado em *chiplets*. Isso significa que a GPU não é uma pastilha monolítica, mas sim composta por vários *dies* (pequenos *chips*) interconectados. O MI300X utiliza oito XCDs (*Accelerator Complex Dies*), que contêm os núcleos de computação reais da GPU, quatro IODs (*I/O Dies*) responsáveis pelo gerenciamento de memória, conectividade e roteamento de dados. Todos esses *dies* são conectados por meio da tecnologia *Infinity Fabric* da AMD, o que permite uma comunicação de alta velocidade e baixa latência. Um esquema da hierarquia de memória da MI300X pode ser visto na Figura 1.8.

A MI300X possui uma interface de memória de 8.192 bits. Essa largura massiva, combinada com a alta frequência da HBM3, resulta em uma lar-

Figura 1.8 – Hierarquia de Memória



gura de banda de memória de pico de 5.3 TB/s (Terabytes por segundo). A MI300X também incorpora um *Last Level Cache* (LLC) de 256 MB. Esse cache ajuda a reduzir a latência de acesso aos dados, armazenando informações frequentemente usadas mais perto dos núcleos de computação. Embora não seja tão proeminente nas especificações diretas da MI300X quanto em outras GPUs da AMD, a arquitetura CDNA 3 se beneficia da tecnologia *Infinity Cache*, que otimiza ainda mais o acesso aos dados e a eficiência.

Essa composição de memória, com a vasta quantidade de HBM3, uma altíssima largura de banda e o projeto modular de *chipllets*, é fundamental para o desempenho da MI300X em tarefas que exigem acesso rápido a grandes volumes de dados, como o treinamento de modelos de IA e simulações de HPC (AMD, 2024).

A MI300X utiliza uma pilha de *software* aberta, denominada ROCm, que inclui *drivers*, ferramentas de desenvolvimento, compiladores e APIs. O ROCm se destaca por ser uma alternativa ao CUDA da NVIDIA, sendo

focado em flexibilidade e transparência².

O ROCm suporta vários modelos de programação, tais como o HIP (*Heterogeneous-compute Interface for Portability*), uma API que facilita a portabilidade de código CUDA para GPUs AMD, o que é um grande atrativo para desenvolvedores que já têm aplicações escritas para a plataforma da NVIDIA.

² <https://rocm.docs.amd.com/en/latest/>

1.4 EXERCÍCIOS

1. Detalhe os diferentes níveis de execução paralela que podem ser explorados em uma aplicação.
2. As arquiteturas paralelas têm como objetivo alcançar níveis diferenciados de desempenho com o uso de múltiplos processadores para a execução em paralelo das aplicações. Com relação a isso, responda:
 - a) Quais são e quais as formas de programação resultantes dos dois grandes tipos de arquiteturas paralelas?
 - b) Quais as características principais das arquiteturas de memória compartilhada?
 - c) Quais as características principais das arquiteturas de memória distribuída?
3. A classificação de Flynn tenta estabelecer uma taxonomia para a subdivisão dos diversos tipos de arquitetura paralela. Com relação a isso, responda:
 - a) Descreva e caracterize brevemente as arquiteturas SISD.
 - b) Descreva e caracterize brevemente as arquiteturas SIMD, exemplificando os seus principais tipos.
 - c) Quais são as características principais das instruções vetoriais e como contribuem para a eficiência em operações sobre conjuntos lineares de valores?
 - d) Cite um exemplo típico de instruções vetoriais encontradas em processadores modernos.
 - e) Descreva e caracterize as arquiteturas MIMD com Memória Distribuída.
 - f) Descreva e caracterize as arquiteturas MIMD de memória compartilhada, diferenciando os seus diversos modelos.
 - g) Quais são os principais desafios de escalabilidade em sistemas UMA com barramento único e como as arquiteturas NUMA modernas buscam mitigar esses problemas?
4. Responda às seguintes questões sobre aceleradores:
 - a) O que são aceleradores e qual seu principal objetivo no contexto da computação de alto desempenho?
 - b) Exemplifique e descreva alguns dispositivos deste tipo.

- c) Explique por que a movimentação de dados é um fator de grande impacto no desempenho das aplicações nos aceleradores.
 - d) Qual o conceito de “kernels” e como são executados nesses dispositivos?
 - e) Explique a organização de um Multiprocessador de Fluxo (SM) em uma arquitetura de GPU como a Hopper, mencionando o escalonador de *warp* e o número de *threads* em um *warp*.
 - f) Considerando a arquitetura Hopper, descreva a hierarquia de memória disponível em cada SM (cache L1, memória compartilhada) e a cache L2, explicando brevemente suas funções e como contribuem para o desempenho.
 - g) Quais são os principais desafios na programação de aceleradores, especialmente no que diz respeito ao gerenciamento de dados entre espaços de memória distintos?
 - h) Como os padrões baseados em diretivas (OpenMP *offloading* e OpenACC) abordam essa questão e como ajudam a gerenciar a computação e a movimentação de dados.
5. Reconhecendo a realidade dos sistemas de alto desempenho modernos, que combinam diferentes tipos de arquiteturas, descreva como esses sistemas são tipicamente compostos.
6. Como a combinação de MPI (para paralelismo entre nós) com OpenMP (para paralelismo dentro dos nós e *offloading* para aceleradores) e OpenACC permite explorar o paralelismo em diferentes níveis e alcançar escalabilidade e alto desempenho?